CASE STUDY 161

Random Password Generator using C++

Name: Aashika Pandey
Roll No:150096725010
Course: B.Tech CSE 2025-29
Subject: C++ Programming (Semester II)
University: ITM Skills University

---

1. Introduction to the Case Study

In today's digital world, strong passwords are very important for protecting personal and online data. Weak passwords can easily be guessed or hacked, which can lead to security problems. To solve this issue, a Random Password Generator can be used to create strong and secure passwords.

This case study focuses on developing a Random Password Generator using the C++ programming language. The program generates passwords using a combination of uppercase letters, lowercase letters, numbers, and special symbols. The user can choose the desired password length, and the program will generate a random password accordingly.

The main goal of this case study is to understand the concepts of random number generation, loops, and string manipulation in C++ while building a simple and useful application.

---

2. Problem Statement / Case Background

Passwords play an important role in securing online accounts and systems. Many users create weak passwords that are easy to guess, which can lead to security risks. Therefore, generating strong and random passwords is essential for improving security.

The problem addressed in this case study is the development of a Random Password Generator using C++. The program should allow the user to enter the password length and generate a random password containing different types of characters.

The generated password should include uppercase letters, lowercase letters, numbers, and symbols to ensure stronger security. The program should also display the generated password to the user.

---

3. Problem Statement / Case Study Design

The Random Password Generator is designed as a simple menu-driven program. The program provides two main options to the user: generating a password or exiting the program.

First, the program displays a menu asking the user to select an option. If the user selects the password generation option, the program asks for the desired password length. The program then generates random characters using predefined character sets.

The password is created using a mixture of uppercase letters, lowercase letters, numbers, and special symbols. Finally, the generated password is displayed on the screen.

This design ensures that the program is simple, user-friendly, and capable of generating secure passwords.

---

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

The program uses basic programming concepts of C++ to generate passwords.

Methods Used

The following concepts are used in this program:

• Random number generation
• String manipulation

• Menu-driven programming
• Loops and conditional statements

Algorithm

Step 1: Start the program.
Step 2: Define character sets for uppercase letters, lowercase letters, numbers, and symbols.
Step 3: Combine all characters into a single string.
Step 4: Initialize the random number generator using srand(time(0)).
Step 5: Display the menu with two options: Generate Password and Exit.
Step 6: Take the user's choice as input.

If the user selects Generate Password:
• Ask for the password length.
• Ensure the password length is at least 4.
• Add one uppercase letter, one lowercase letter, one number, and one symbol.
• Use a loop to add remaining random characters.
• Display the generated password.

If the user selects Exit:
• Terminate the program.

Step 7: End the program.

---

5. Problem Statement / Case Study Implementation Details and Snapshots

The Random Password Generator program is implemented using the C++ programming language. The program uses standard libraries such as iostream, cstdlib, and ctime to perform input/output operations and random number generation.

The user first selects the option to generate a password. The program then asks the user to enter the desired password length. After receiving the input, the program generates random characters from predefined character sets.

The generated password includes uppercase letters, lowercase letters, numbers, and special symbols, ensuring better security.

randompass > random.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
int main()
{
    int choice, length;
    string upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    string lower = "abcdefghijklmnopqrstuvwxyz";
    string numbers = "0123456789";
    string symbols = "@#$%&*";
    string all = upper + lower + numbers + symbols;
    srand(time(0));
    while (true)
    {
        cout << "\n----- Random Password Generator -----\n";
        cout << "1. Generate Password\n";
        cout << "2. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        if (choice == 1)
        {
            cout << "Enter password length: ";
            cin >> length;
            if (length < 4)
            {
                cout << "Password length should be at least 4\n";
                continue;
            }
            string password = "";
            // Ensure one of each type
            password += upper[rand() % upper.length()];
            password += lower[rand() % lower.length()];
            password += numbers[rand() % numbers.length()];
            password += symbols[rand() % symbols.length()];
            for (int i = 4; i < length; i++)
            {
                password += all[rand() % all.length()];
            }
            cout << "Generated Password: " << password << endl;
        }
        else if (choice == 2)
        {
            cout << "Exiting program...\n";
            break;
        }
        else
        {
            cout << "Invalid choice. Try again.\n";
        }
    }
    return 0;
}
```

```
----- Random Password Generator -----
1. Generate Password
2. Exit
Enter your choice: 1
Enter password length: 8
Generated Password: Px9$RYDi
```

---

6. Problem Statement / Case Study Results and Conclusion

The Random Password Generator program successfully generates strong and secure passwords using different types of characters. The program allows the user to select the password length and generates a random password accordingly.

The use of random number generation ensures that each password generated is different and difficult to guess. The program also demonstrates the use of loops, conditional statements, and string manipulation in C++.

In conclusion, this case study helped in understanding the concepts of randomization, secure password creation, and menu-driven programming in C++. The developed program can be useful for generating strong passwords to improve security.

---

7. References

1. Stroustrup, B. *The C++ Programming Language*.
2. https://cplusplus.com
3. https://www.geeksforgeeks.org
4. ITM Skills University – Case Study 161 Problem Statement.

---

10. GiHub Link

CASE STUDY 161

---

11. Presentation Link

Ppt